Table of Contents

1.0 Abstract 1

2.0 Discussion 2

2.1 Objective 2

2.2 Target Users 2

2.3 Background of the Design 2

3.0 Design 3

4.0 Implementation 6

4.1 IWS Model Implementation 6

4.2 Key Control Implementation 10

4.3 Climate Mode Operation 12

4.4 Advanced Features 14

4.5 Technical Implementation Detail 15

5.0 Problems and Objectives 16

6.0 The Screen Layout 19

References 22

APPENDIX 1 24

APPENDIX 2 51

Abstract

Intelligent Window Systems (IWS) are smart architectural products designed to enhance indoor comfort and reduce energy consumption by dynamically controlling light transmission, airflow, and ventilation. Early window concepts focused on passive energy management, primarily reducing heat gain and improving natural ventilation. With advancements in materials, electronics, and automation, modern IWS have become more advance, integrating electrochromic smart glass, motorized vents, and environmental sensors. These systems are now widely used in residential and commercial buildings to optimize comfort and sustainability.

This project innovates based on these functional characteristics, designing a responsive window system that adjusts glass transparency, opens or closes vents, and operates fans automatically according to temperature, sunlight, and time of day. A 3D visualization was created using OpenGL in CodeBlocks, with C language hierarchical modeling techniques to construct individual components. The outer smart glass layer, solar control coating, airflow chamber with micro vents, inner insulated glass, and embedded frame system are represented with distinct textures, transparency, and colors. Animation simulates vent movement, fan rotation, airflow, and sensor indicators, demonstrating realistic behavior. This approach illustrates how an IWS can improve energy efficiency by reducing reliance on air-conditioning and artificial lighting

Discussion

1. Objective

With increasing global temperatures and rising energy consumption in modern buildings, maintaining indoor comfort efficiently has become a challenge. Traditional fixed windows and basic ventilation systems are no longer sufficient to meet the cooling and lighting needs of contemporary homes and offices. This project addresses the issue by designing an Intelligent Window System (IWS) that can automatically adjust light transmission, airflow, and ventilation in response to environmental conditions. The goal is to provide a smart, energy-saving solution that reduces the cost of using air-conditioning, makes indoor spaces more comfortable for people and helps save energy, supporting a more environmentally friendly way of living.

2. Target Users

The main users for the Intelligent Window System (IWS) include residential homeowners who want to reduce their electricity bills and live in a more comfortable, energy-efficient home. Commercial building owners can also benefit, as the system helps lower operational costs while maintaining a pleasant indoor environment for employees and visitors. Additionally, architects, developers, and designers working on smart buildings or green architecture projects can incorporate this system to enhance sustainability and meet modern energy efficiency standards. Generally, anyone looking for smarter, adaptive solutions to manage indoor climate and reduce energy consumption could benefit from this technology.

3. Background of the Design

Modern buildings use a lot of energy to keep spaces cool and well-lit, especially in areas with hot climates. Traditional windows, however, are passive. They let in light and air but cannot adapt to changing conditions like temperature or sunlight. This design introduces intelligence into a basic architectural element, turning a standard window into an active system that automatically controls light, airflow, and ventilation. By responding to environmental changes in real time, the Intelligent Window System helps reduce energy use, improves indoor comfort, and supports a more sustainable approach to building design.

Design

The idea for the Intelligent Window System (IWS) emerged from observing how many modern buildings rely on static windows that cannot adapt to changing environmental conditions, often leading to unnecessary energy consumption for heating, cooling, or lighting. I wanted to create a solution that combines smart glass technology with adaptive airflow control, allowing windows to respond automatically to real-time temperature, sunlight, and ventilation needs. This approach not only improves energy efficiency but also enhances indoor comfort and visual aesthetics.

My interest in this concept was further inspired by a personal experience. During a visit to an electrochromic window factory in China with my father, I had the opportunity to see smart glass technology and learn about its applications, limitations, and potential in modern buildings. Experiencing how the glass changes tint in response to voltage and sunlight give me the idea to elaborate on this concept.

The Intelligent Window System (IWS) is a modern window design that combines smart glass technology with controlled airflow to improve indoor comfort and save energy. Unlike traditional windows, which only provide passive insulation or sunlight control, the IWS actively responds to environmental conditions like sunlight intensity, temperature changes, and indoor climate needs.

The system architecture progresses from exterior to interior as follows:

Outer Smart Glass Layer → Solar Control & Coating Layer → Airflow Chamber + Micro Vents → Inner Insulated Glass Layer → Embedded Frame System

This layered design allows the window to manage light, heat, and ventilation independently, making it both energy-efficient and comfortable.

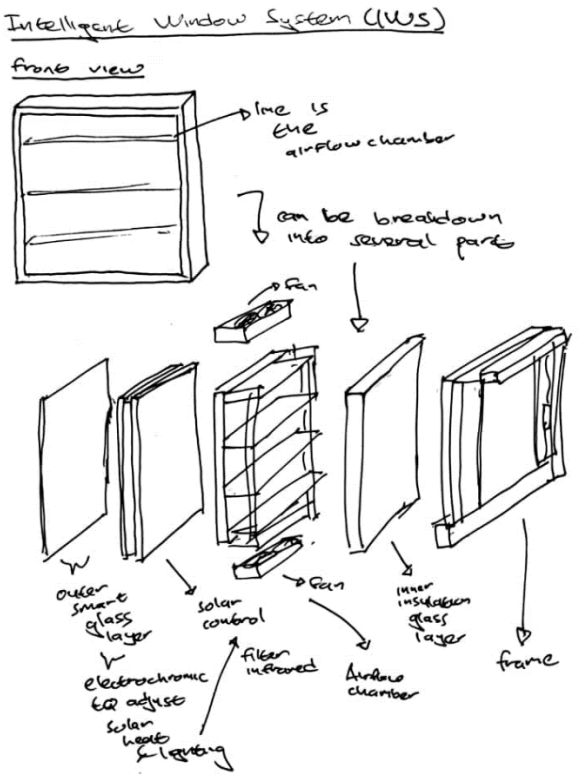


Figure 1 Hand Drawn Concept

Detailed Component Design:

A. Outer Smart Glass Layer

The outer layer of the IWS uses smart glass that can change its transparency depending on sunlight and temperature. This allows the window to reduce glare, control heat, and maintain natural lighting indoors. The glass uses electrochromic technology, which adjusts automatically when voltage is applied. Typical transitions take between 3–20 minutes. Studies have shown electrochromic windows can reduce primary energy use by 10-20% in most climates compared to conventional glazing (Lee, Yazdanian, & Selkowitz, 2004; U.S. Department of Energy, 2022).

B. Solar Control and Coating Layer

Beneath the smart glass is a solar control layer, which reflects unwanted heat from sunlight while allowing visible light in. This layer helps maintain indoor comfort without

blocking natural light. It is made with thin-film coatings that efficiently block infrared radiation, reducing solar heat gain while preserving a bright environment (Karlsson, 2001).

C. Adaptive Airflow Chamber and Micro Vents

The airflow chamber is the core innovation of the IWS. It allows air to move between the outer and inner glass layers, which can cool the room naturally or with small fans when needed. This design mimics double-skin façade principles used in modern buildings. The vents can open for passive cooling, or small brushless fans can assist in extreme heat. This approach can lower indoor cooling loads by 25–35% through natural ventilation and thermal buffer effects (Ahmed et al., 2016; Saifi et al., 2023).

D. Inner Insulated Glass Layer

The inner layer is a standard low-emissivity (low-E) glass that improves insulation and soundproofing. It reduces heat loss in cold weather and prevents heat from entering in hot weather. Combined with the outer layers, it helps maintain a stable and consistent indoor climate. Low-E glass coatings can reduce heat transfer by up to 30-50% compared to standard glazing (Low-E glass impact on thermal comfort, 2025).

E. Embedded Frame System

The frame houses all control systems, including sensors, motors, and power supply. Temperature, sunlight, and airflow sensors feed data to a microcontroller, which automatically adjusts glass tint and vent positions. This smart frame ensures the IWS responds efficiently to real-time environmental changes and saves energy. Microcontroller-based automation systems enable precise environmental control and optimization of building energy performance (Khan et al., 2019).

Implementation

Through CodeBlocks, we create a 3D scene primarily used to display a model that simulates an Intelligent Window System. The code utilizes the OpenGL library and the GLUT toolkit for graphics rendering. We have implemented objects such as smart glass layers, solar coating, airflow chambers, cooling fans, micro vents, sensors, and an intelligent frame system. Here are the details of the implementation.

1. IWS Model Implementation

Outer Smart Glass Layer

This part creates and textures the outermost glass layer of the window system. Use GL_TEXTURE_2D to enable textures and create quadrangles with glBegin(GL_QUADS) and glEnd(). The glass texture is procedurally generated with a smooth shiny effect applied to the quadrilateral. The tint level dynamically adjusts based on climate mode, changing both color and transparency. Finally, use GL_LINE_LOOP to draw a border with glow effect to enhance the visual appearance.

Solar Coating Layer

This part constructs the solar energy harvesting coating layer positioned just behind the outer glass. Draw the coating in the form of GL_QUADS with semi-transparent orange-yellow color to simulate the photovoltaic coating. The layer is positioned at -1.5 units in normal view and separates further during exploded view mode.

Airflow Chamber

This part creates the central airflow chamber structure between glass layers. It is implemented by using GL_QUADS to draw the top, bottom, left, and right walls of the chamber. The chamber includes animated airflow visualization using curved lines (GL_LINE_STRIP) that change color based on climate mode: red for hot mode (heat exhaust), blue for cold mode (cold air circulation), and green for normal mode. The curved lines simulate air movement using sine wave calculations.

Micro Vents

This part builds the adjustable ventilation system at the top and bottom of the window. Eight individual vent slats are rendered at each location using GL_QUADS. Each vent can rotate using glRotatef() with the ventAngle parameter, controlled by the vent animation system. The vents open (90°) in hot mode and close (0°) in cold mode, with intermediate positions for normal mode.

Mini Cooling Fans

This part creates and renders the dual cooling fan system. The fan housing is constructed using GL_QUADS with metal texture mapping applied via glBindTexture(GL_TEXTURE_2D, metalTexture). A ventilation grill pattern is drawn on the front face using GL_LINES to create a realistic industrial appearance. Four fan blades are rendered for each fan using multiple GL_QUADS to create 3D blade geometry. The blades rotate continuously using glRotatef(angle, 0, 0, 1) with speed controlled by fanSpeedMultiplier. A central hub is created using gluCylinder() and gluDisk() for the motor assembly.

Inner Insulated Glass Layer

This part constructs the innermost glass layer that provides thermal insulation. Similar to the outer glass, it uses GL_QUADS with glass texture mapping but with slightly different color tinting (more neutral blue-gray) and increased thickness (1.5 units) to represent the insulation properties. The layer is positioned at -10 units in normal view.

Intelligent Frame System

This part builds the structural frame and control system housing. Four frame sections (top, bottom, left, right) are constructed using glutSolidCube() with metal texture mapping for a brushed aluminum appearance. The frame contains multiple subsystems:

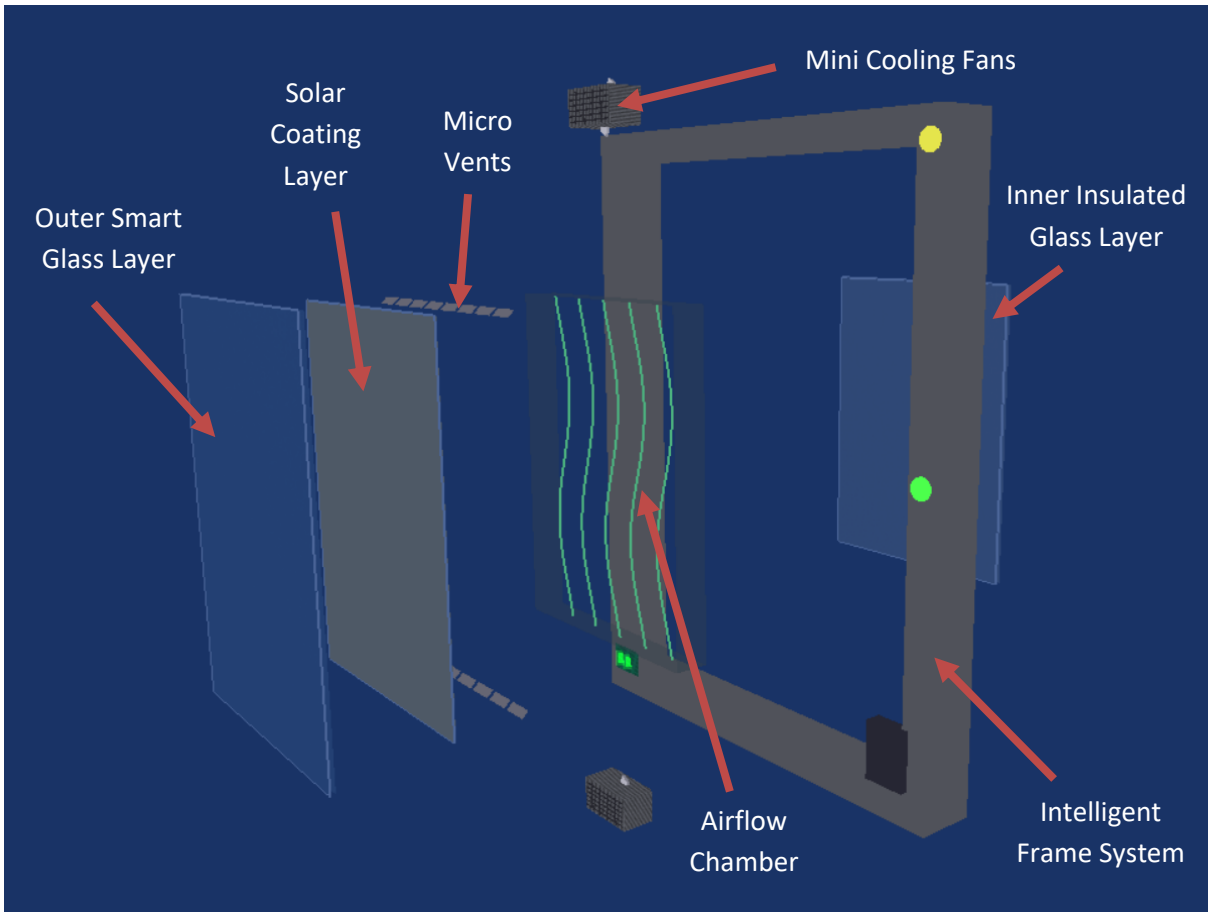
Climate Sensors: Rendered using glutSolidSphere() with color-coded indicators - red for hot mode detection, blue for cold mode detection, and green for normal mode.

Light Sensor: A yellow sphere positioned at the top right to detect ambient light levels.

Microcontroller Box: A dark rectangular box created with glutSolidCube() to represent the control electronics.

Display Panel: A small digital screen rendered with GL_QUADS and custom display texture showing a grid pattern to simulate an LCD interface.

LED Indicators: Pulsating green point lights rendered using GL_POINTS with dynamic glow effect. The glow uses blending mode glBlendFunc(GL_SRC_ALPHA, GL_ONE) and animates using sin(animationTime * 3.0f) to create a breathing effect.



Procedural Texture Generation

Three procedural textures are generated at initialization:

Metal Texture: Creates a brushed metal effect using noise and horizontal sine waves to simulate the appearance of brushed aluminum. Implemented in createMetalTexture() with 256×256 resolution.

Glass Texture: Generates a smooth, shiny glass appearance with subtle noise and gradient for realistic reflection. Implemented in createGlassTexture() with RGBA format for transparency.

Display Texture: Creates a digital display grid pattern to simulate an LCD screen. Implemented in createDisplayTexture() with darker background and lighter grid lines.

2. Key Control Implementation

Special Keys (using the specialKeys function):

GLUT_KEY_RIGHT: Increases the Y-axis rotation angle for horizontal view rotation.

GLUT_KEY_LEFT: Decreases the Y-axis rotation angle for horizontal view rotation.

GLUT_KEY_UP: Increases the X-axis rotation angle for vertical view rotation.

GLUT_KEY_DOWN: Decreases the X-axis rotation angle for vertical view rotation.

Mouse Control (using mouse and mouseMotion functions):

Press and move left mouse button: Adjust the rotation angle of the X and Y axes according to the distance of the mouse movement, allowing free 360° rotation of the view.

Common Key Control (using the keyboard function):

Camera Controls:

'z'/'Z': Zoom in (reduce camera distance, minimum 50 units).

'x'/'X': Zoom out (increase camera distance, maximum 300 units).

'a'/'A': Rotate view counterclockwise around Z-axis.

'd'/'D': Rotate view clockwise around Z-axis.

Lighting Controls:

'l'/'L': Turn on lighting (lightEnabled = true).

'k'/'K': Turn off lighting (lightEnabled = false).

Component Controls:

'f'/'F': Toggle top cooling fan rotation (topFanRotate).

'g'/'G': Toggle bottom cooling fan rotation (bottomFanRotate).

'v'/'V': Toggle vent animation (ventAnimate) - vents will open and close automatically.

't'/'T': Increase glass tint level (darker, maximum 1.0).

'**y**'/'**Y**': Decrease glass tint level (clearer, minimum 0.0).

Climate Mode Controls:

'**1**': Activate HOT mode (dark glass tint 0.8, vents fully open 90°, fans at 2× speed).

'**2**': Activate COLD mode (clear glass tint 0.1, vents closed 0°, fans off).

'**3**': Activate NORMAL mode (medium glass tint 0.4, vents half-open 45°, fans at normal speed).

Exploded View Controls:

'**e**'/'**E**': Toggle exploded view mode (separates all layers for inspection).

'+'/'=': Increase explosion distance (maximum 150 units).

'-'/'_': Decrease explosion distance (minimum 0 units, returns to normal view).

Help & Exit:

'**h**'/'**H**': Display complete control help in console.

ESC (27): Exit the program.

3. Climate Mode Operation

HOT Mode Operation:

When the user presses **'1'**:

- Glass tint darkens to 0.8 (80% opacity) to block heat and sunlight.
- Micro vents open fully to 90° to allow hot air exhaust.
- Both cooling fans activate and spin at 2× normal speed for maximum cooling.
- Climate sensor displays red color indicating hot temperature detection.
- Airflow visualization shows red curved lines representing hot air movement.
- Fan speed multiplier set to 2.0 for aggressive heat management.

COLD Mode Operation:

When the user presses **'2'**:

- Glass tint clears to 0.1 (90% transparent) to allow maximum sunlight and passive solar heating.
- Micro vents close completely to 0° to prevent heat loss and cold air infiltration.
- Both cooling fans stop rotating to conserve energy and prevent cold air circulation.
- Climate sensor displays blue color indicating cold temperature detection.
- Airflow visualization shows blue curved lines representing minimal air movement.
- Fan speed multiplier set to 0.0 (fans disabled).

NORMAL Mode Operation:

When the user presses **'3'**:

- Glass tint adjusts to moderate level 0.4 (60% transparent) for balanced light control.
- Micro vents open partially to 45° for moderate ventilation.
- Both cooling fans rotate at normal 1× speed for standard air circulation.

- Climate sensor displays green color indicating comfortable temperature.
- Airflow visualization shows green curved lines representing balanced airflow.
- Fan speed multiplier set to 1.0 for standard operation.

Manual Fine-Tuning:

Users can override preset modes with manual controls:

- Press **'t'/'y'** repeatedly to incrementally adjust glass tint in 0.1 steps.
- Press **'v'** to activate automatic vent breathing animation (open/close cycle).
- Press **'f'** or **'g'** individually to control top or bottom fans independently.

4. Advanced Features

Exploded View Mode

The exploded view separates all layers along the Z-axis to reveal the internal structure:

- Outer smart glass moves forward by explosionDistance × 1.5.
- Solar coating moves forward by explosionDistance × 1.2.
- Airflow chamber moves forward by explosionDistance × 0.5.
- Micro vents move forward by explosionDistance × 1.0.
- Top fan moves up by explosionDistance × 1.0 and forward by explosionDistance × 0.5.
- Bottom fan moves down by explosionDistance × 1.0 and forward by explosionDistance × 0.5.
- Inner glass moves backward by explosionDistance × 0.8.
- Intelligent frame scales outward by 1.0 + explosionDistance × 0.01.

Users can adjust explosion distance dynamically using **+/-** keys for detailed component inspection.

Animation System

A continuous timer callback (timer function) runs at 60 FPS (16ms intervals) to handle:

- Fan blade rotation with smooth angle incrementation.
- Vent opening/closing animation with automatic direction reversal.
- LED indicator pulsation using sinusoidal glow intensity.
- Global animation time tracking for synchronized effects.

Real-time Visual Feedback

The system provides immediate visual feedback for all operations:

- On-screen HUD displays current mode (HOT/COLD/NORMAL) and exploded view status.

- Console output confirms mode changes and control activations.
- Color-coded sensors provide instant visual climate indication.
- Pulsating LEDs show system active status.
- Airflow visualization dynamically represents current thermal management state.

5. Technical Implementation Details

Graphics Pipeline:

- Uses double buffering (GLUT_DOUBLE) for smooth animation.
- Depth testing (GL_DEPTH_TEST) for proper 3D rendering.
- Alpha blending (GL_BLEND) for transparent glass effects.
- Texture mapping with mipmapping for high-quality surface details.

Problems & objectives

Problem 1 (Exploded View Positioning Calculations)

The first issue we encountered was when Implementing the exploded view feature required careful calculation of each component's offset position. Initially, we used uniform explosion distances for all components, which resulted in poor visual separation—some components would overlap while others would be too far apart. The solution to this is we implemented proportional explosion offsets to each component's size

```
// Outer glass - maximum separation
if(explodedView) glTranslatef(0, 0, explosionDistance * 1.5f);
// Solar coating - slightly less
if(explodedView) glTranslatef(0, 0, explosionDistance * 1.2f);
// Airflow chamber - middle position
if(explodedView) glTranslatef(0, 0, explosionDistance * 0.5f);
// Inner glass - backward separation
if(explodedView) glTranslatef(0, 0, -explosionDistance * 0.8f);
```

Different multipliers (1.5×, 1.2×, 0.5×, -0.8×) create visually balanced spacing that clearly shows the layer structure without excessive separation. The frame also scales slightly outward to accommodate the expanded layers.

Problem 2 (Glass Transparency and Layer Rendering)

Another issue occurred when rendering multiple transparent glass layers. Initially, when all glass layers were rendered in arbitrary order, the transparency blending produced incorrect visual results. Layers in front would sometimes obscure layers behind them completely, or transparency would not blend properly, creating a confusing visual appearance where users could not see through the multi-layered window structure properly.

We speculated that the cause of this phenomenon was due to incorrect rendering order and improper alpha blending configuration. In OpenGL, transparent objects must be rendered

in a specific order (back to front) for proper blending. Without proper setup, the depth buffer would reject fragments that should be visible through transparent surfaces.

To address this problem, we implemented proper transparency handling through enabling alpha blending with `glEnable(GL_BLEND)` and `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. We also ensured correct layer ordering by rendering glass layers from back to front: inner insulated glass (furthest back at Z = -10), solar coating (Z = -1.5), and outer smart glass (closest to viewer). Additionally, we implemented dynamic alpha values that change with tint level using `float tintAlpha = 0.6f - (glassTintLevel * 0.4f)`. This ensures realistic glass appearance with proper see-through effects and correct visual layering throughout all climate modes.

Problem 3 (Vent Animation Direction Reversal)

Another challenge occurred when implementing the automatic vent opening and closing animation. The vents needed to smoothly open from 0° to 90° and then reverse direction to close back to 0° in a continuous breathing pattern. Initially, we attempted to use a simple increment without direction tracking, which caused the vents to either overshoot beyond 90° or get stuck at the boundaries.

To address this we introduced a boolean flag ventOpening to track the current animation direction. The animation logic checks the current angle and reverses direction when boundaries are reached:

```
if(ventAnimate) {
   if(ventOpening) {
      ventAngle += 2.0f;
      if(ventAngle >= 90.0f) {
         ventAngle = 90.0f;
         ventOpening = false;
      }
   } else {
```

```
            ventAngle -= 2.0f;
            if(ventAngle <= 0.0f) {
                ventAngle = 0.0f;
                ventOpening = true;
            }
        }
    }
```

This state-machine approach ensures the vents animate smoothly between fully closed and fully open positions without overshooting or getting stuck.

The screen layout

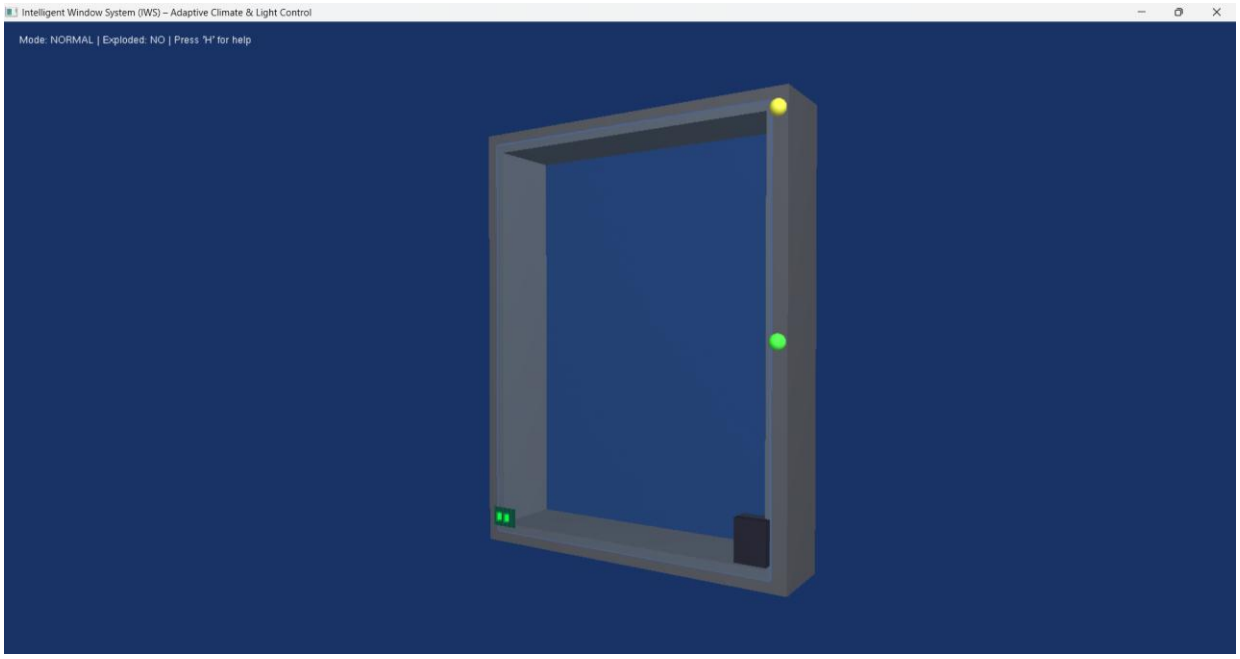


Figure 2 Assembled View

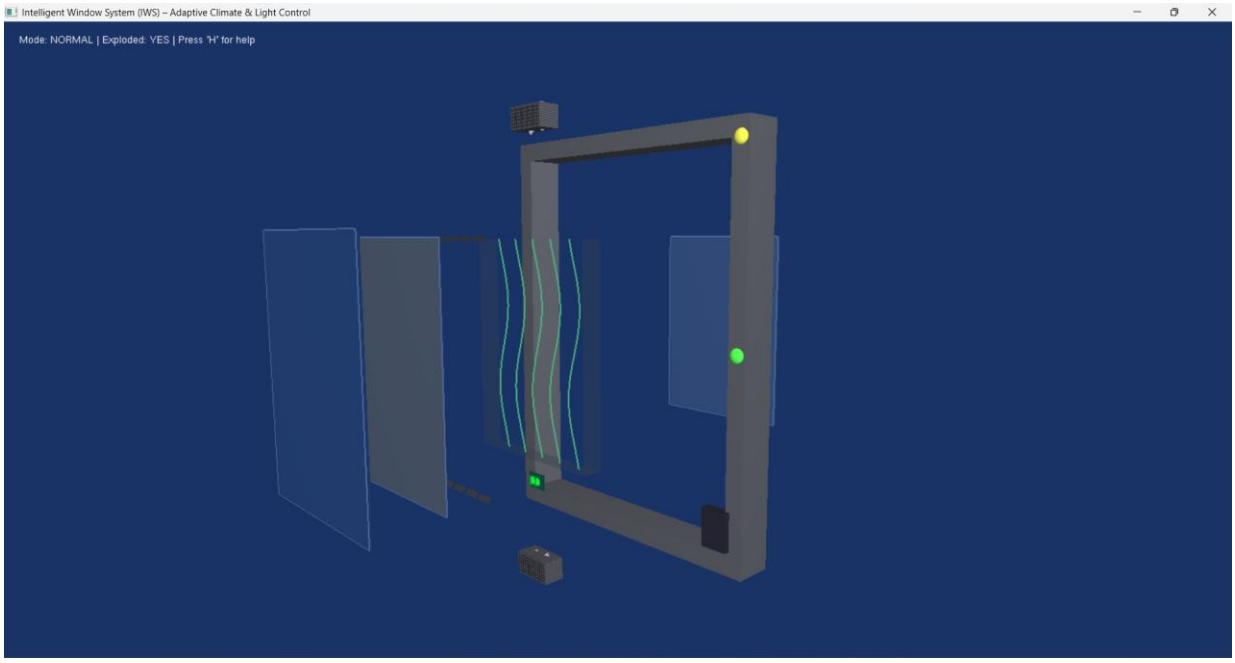


Figure 3 Exploded View

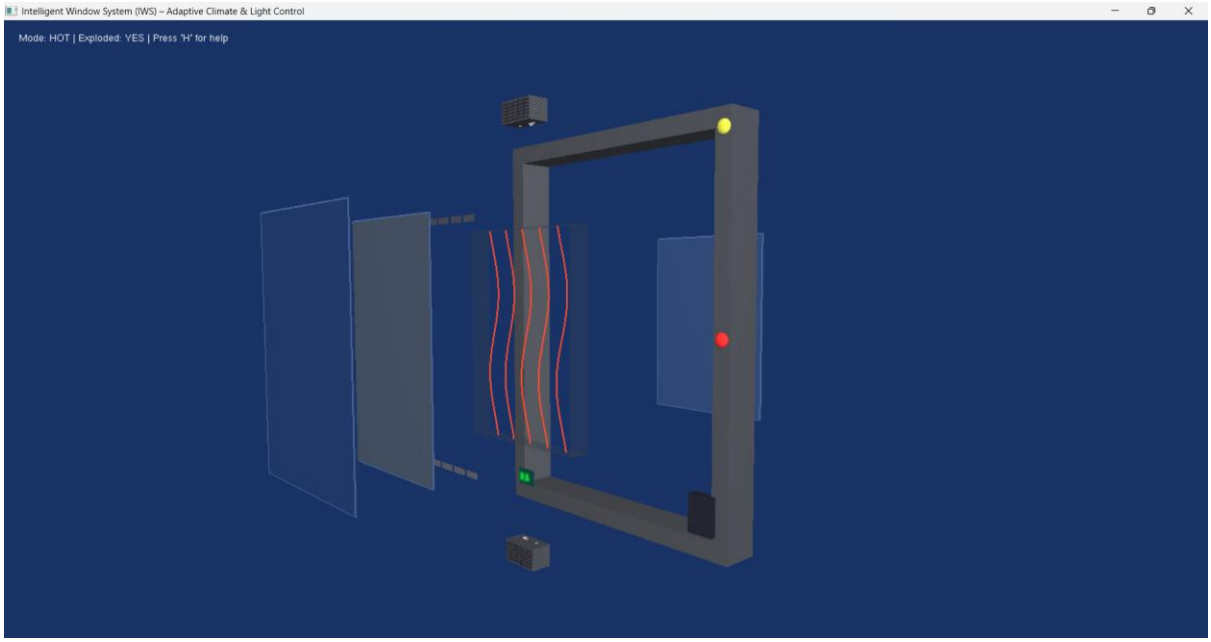


Figure 4 Mode 1 (HOT): Dark glass, open vents, fast fans

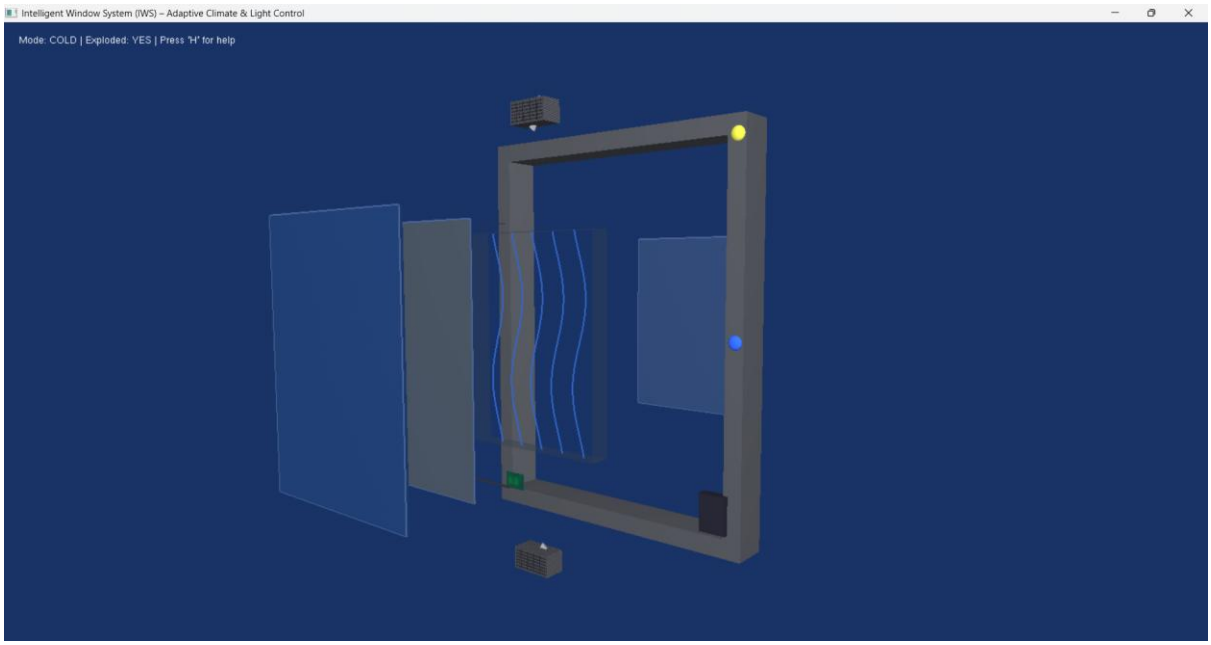


Figure 5 Mode 2 (COLD): Clear glass, closed vents, fans off

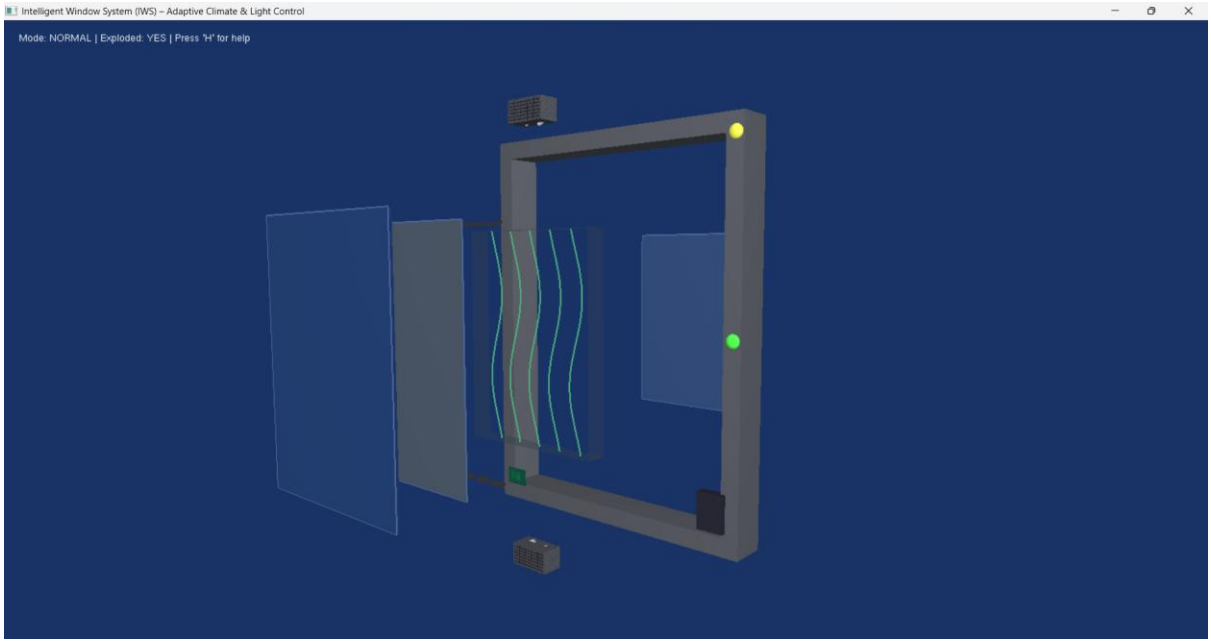


Figure 6 Mode 3 (NORMAL): Balanced settings

Reference

Ahmed, M. M., Abel-Rahman, A. K., & Ali, A. H. H. (2016). Double skin façade: The state of art on building energy efficiency. *Journal of Clean Energy Technologies*, *4*(1), 84-89. https://doi.org/10.7763/JOCET.2016.V4.258

Guardian Glass. (n.d.). *Low-e glass: Low emissivity & energy efficiency glass coating*. https://www.guardianglass.com/us/en/our-glass/glass-types/low-e-glass

Karlsson, J. (2001). *Windows – Optical performance and energy efficiency* [Doctoral dissertation, Uppsala University]. DiVA Portal. https://www.diva-portal.org/smash/get/diva2:161001/FULLTEXT01.pdf

Khan, T. Z., Adhikary, A., & Khan, M. A. R. (2019). Microcontroller based industrial automation system using temperature sensor and output logic control. *International Journal of Computer Applications*, *178*(43), 40-43. https://doi.org/10.5120/ijca2019919326

Lee, E. S., Yazdanian, M., & Selkowitz, S. E. (2004). *The energy-savings potential of electrochromic windows in the US commercial buildings sector* (Report No. LBNL-54966). Lawrence Berkeley National Laboratory. https://bies.lbl.gov/publications/energy-savings-potential

Low-E Glass Impact on Thermal Comfort and Energy Savings. (2025). *Industrial Energy Network.* https://www.ien.com.my/post/low-e-glass-impact-on-thermal-comfort-and-energy-savings

Saifi, N., Settou, N., Dokkar, B., Negrou, B., & Chennouf, N. (2023). The role of double-skin facade configurations in optimizing building energy performance in Erbil city. *Scientific Reports*, *13*, 8303. https://www.nature.com/articles/s41598-023-35555-0

U.S. Department of Energy. (2022). Better windows, better outcomes: How electrochromics improve health, productivity, and efficiency. Building Technologies Office. https://www.energy.gov/sites/default/files/2023-12/bto-electrochromic-window-report-121123.pdf

U.S. Department of Energy. (n.d.). Window types and technologies. https://www.energy.gov/energysaver/window-types-and-technologies

Vitro Architectural Glass. (2025). *How low-e glass works*. https://glassed.vitroglazings.com/topics/how-low-e-glass-works

APPENDIX 1

CODE

```
#include <windows.h>
#include <GL/glut.h>
#include <math.h>
#include <stdio.h>

// Rotation and Camera
float xRotation = 0.0f, yRotation = 0.0f, zRotation = 0.0f;
float cameraDistance = 150.0f;
int lastMouseX, lastMouseY;
bool isMouseDragging = false;

// Component States
bool lightEnabled = true;
bool topFanRotate = false;
bool bottomFanRotate = false;
bool ventAnimate = false;
float topFanAngle = 0.0f;
float bottomFanAngle = 0.0f;
float ventAngle = 0.0f;
bool ventOpening = true;

// Glass and Climate States
float glassTintLevel = 0.3f;
int climateMode = 3; // Hot, Cold, Normal

// Exploded View
bool explodedView = false;
float explosionDistance = 0.0f;
float maxExplosion = 80.0f;

// Animation
```

```
float fanSpeedMultiplier = 1.0f;
float animationTime = 0.0f;

// Texture IDs
GLuint metalTexture, glassTexture, displayTexture;

// Function to create metal texture
GLuint createMetalTexture() {
    const int texSize = 256;
    unsigned char* data = new unsigned char[texSize * texSize * 3];

    for(int i = 0; i < texSize; i++) {
        for(int j = 0; j < texSize; j++) {
            int idx = (i * texSize + j) * 3;
            // Create brushed metal effect
            float noise = (rand() % 30) / 255.0f;
            float horizontal = sin(j * 0.5f) * 0.1f;
            float baseColor = 0.4f + noise + horizontal;
            data[idx] = (unsigned char)(baseColor * 200);
            data[idx + 1] = (unsigned char)(baseColor * 205);
            data[idx + 2] = (unsigned char)(baseColor * 210);
        }
    }

    GLuint texID;
    glGenTextures(1, &texID);
    glBindTexture(GL_TEXTURE_2D, texID);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, texSize, texSize, 0, GL_RGB,
GL_UNSIGNED_BYTE, data);
    delete[] data;
```

```
    return texID;
}

// Function to create shiny glass texture
GLuint createGlassTexture() {
    const int texSize = 256;
    unsigned char* data = new unsigned char[texSize * texSize * 4];

    for(int i = 0; i < texSize; i++) {
        for(int j = 0; j < texSize; j++) {
            int idx = (i * texSize + j) * 4;
            // Create smooth shiny glass
            float baseNoise = (rand() % 8) / 255.0f;
            float gradient = 0.95f + (float)(i + j) / (texSize * 4) * 0.05f;
            float brightness = gradient + baseNoise;
            if(brightness > 1.0f) brightness = 1.0f;
            data[idx] = (unsigned char)(235 * brightness);
            data[idx + 1] = (unsigned char)(245 * brightness);
            data[idx + 2] = (unsigned char)(255 * brightness);
            data[idx + 3] = 85;
        }
    }

    GLuint texID;
    glGenTextures(1, &texID);
    glBindTexture(GL_TEXTURE_2D, texID);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, texSize, texSize, 0, GL_RGBA,
GL_UNSIGNED_BYTE, data);
    delete[] data;
    return texID;
```

```
}

// Function to create display screen texture
GLuint createDisplayTexture() {
    const int texSize = 128;
    unsigned char* data = new unsigned char[texSize * texSize * 3];

    for(int i = 0; i < texSize; i++) {
        for(int j = 0; j < texSize; j++) {
            int idx = (i * texSize + j) * 3;
            // Create digital display pattern
            float grid = ((i % 4 == 0) || (j % 4 == 0)) ? 0.9f : 0.7f;
            data[idx] = (unsigned char)(20 * grid);
            data[idx + 1] = (unsigned char)(100 * grid);
            data[idx + 2] = (unsigned char)(80 * grid);
        }
    }

    GLuint texID;
    glGenTextures(1, &texID);
    glBindTexture(GL_TEXTURE_2D, texID);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, texSize, texSize, 0, GL_RGB,
GL_UNSIGNED_BYTE, data);
    delete[] data;
    return texID;
}

void init() {
    glClearColor(0.1f, 0.2f, 0.4f, 1.0f);
    glEnable(GL_DEPTH_TEST);
```

```
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT_AND_BACK, GL_AMBIENT_AND_DIFFUSE);

    // Setup lighting
    GLfloat lightPos[] = {50.0f, 100.0f, 100.0f, 1.0f};
    GLfloat lightAmbient[] = {0.3f, 0.3f, 0.3f, 1.0f};
    GLfloat lightDiffuse[] = {0.8f, 0.8f, 0.8f, 1.0f};
    GLfloat lightSpecular[] = {1.0f, 1.0f, 1.0f, 1.0f};
    glLightfv(GL_LIGHT0, GL_POSITION, lightPos);
    glLightfv(GL_LIGHT0, GL_AMBIENT, lightAmbient);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, lightDiffuse);
    glLightfv(GL_LIGHT0, GL_SPECULAR, lightSpecular);

    glEnable(GL_NORMALIZE);
    glEnable(GL_TEXTURE_2D);

    // Create textures
    metalTexture = createMetalTexture();
    glassTexture = createGlassTexture();
    displayTexture = createDisplayTexture();
}

// Draw glass panel
void drawGlassPanel(float width, float height, float depth, float r, float g, float b, float alpha,
bool useTexture) {
    glPushMatrix();
    glColor4f(r, g, b, alpha);

    if(useTexture) {
        glEnable(GL_TEXTURE_2D);
```

```
    glBindTexture(GL_TEXTURE_2D, glassTexture);
}

// Front face
glBegin(GL_QUADS);
glNormal3f(0, 0, 1);
glTexCoord2f(0, 0); glVertex3f(-width/2, -height/2, depth/2);
glTexCoord2f(1, 0); glVertex3f(width/2, -height/2, depth/2);
glTexCoord2f(1, 1); glVertex3f(width/2, height/2, depth/2);
glTexCoord2f(0, 1); glVertex3f(-width/2, height/2, depth/2);
glEnd();

// Back face
glBegin(GL_QUADS);
glNormal3f(0, 0, -1);
glTexCoord2f(0, 0); glVertex3f(-width/2, -height/2, -depth/2);
glTexCoord2f(0, 1); glVertex3f(-width/2, height/2, -depth/2);
glTexCoord2f(1, 1); glVertex3f(width/2, height/2, -depth/2);
glTexCoord2f(1, 0); glVertex3f(width/2, -height/2, -depth/2);
glEnd();

if(useTexture) glDisable(GL_TEXTURE_2D);

// Edges with glow effect
glDisable(GL_LIGHTING);
glColor3f(0.3f, 0.4f, 0.6f);
glLineWidth(2.0f);
glBegin(GL_LINE_LOOP);
glVertex3f(-width/2, -height/2, depth/2);
glVertex3f(width/2, -height/2, depth/2);
glVertex3f(width/2, height/2, depth/2);
glVertex3f(-width/2, height/2, depth/2);
glEnd();
if(lightEnabled) glEnable(GL_LIGHTING);
```

```
    glPopMatrix();
}

// Draw Outer Smart Glass Layer
void drawOuterSmartGlass() {
    glPushMatrix();
    if(explodedView) glTranslatef(0, 0, explosionDistance * 1.5f);
    glScalef(1.04f, 1.04f, 1.0f);
    float tintAlpha = 0.6f - (glassTintLevel * 0.4f);
    drawGlassPanel(60, 80, 1, 0.3f + glassTintLevel*0.3f, 0.6f, 0.9f - glassTintLevel*0.3f,
tintAlpha, true);
    glPopMatrix();
}

// Draw Solar Coating Layer
void drawSolarCoating() {
    glPushMatrix();
    if(explodedView) {
        glTranslatef(0, 0, explosionDistance * 1.2f);
    } else {
        glTranslatef(0, 0, -1.5f);
    }
    drawGlassPanel(59, 79, 0.3f, 0.8f, 0.7f, 0.4f, 0.3f, false);
    glPopMatrix();
}

// Airflow visualization
void drawAirflowChamber() {
    glPushMatrix();
    if(explodedView) {
        glTranslatef(0, 0, explosionDistance * 0.5f);
    } else {
        glTranslatef(0, 0, -5);
```

```
}

glDisable(GL_LIGHTING);
glColor4f(0.2f, 0.25f, 0.3f, 0.4f);
float w = 58, h = 78, d = 8;

// Top wall
glBegin(GL_QUADS);
glVertex3f(-w/2, h/2, d/2);
glVertex3f(w/2, h/2, d/2);
glVertex3f(w/2, h/2, -d/2);
glVertex3f(-w/2, h/2, -d/2);
glEnd();

// Bottom wall
glBegin(GL_QUADS);
glVertex3f(-w/2, -h/2, d/2);
glVertex3f(-w/2, -h/2, -d/2);
glVertex3f(w/2, -h/2, -d/2);
glVertex3f(w/2, -h/2, d/2);
glEnd();

// Left wall
glBegin(GL_QUADS);
glVertex3f(-w/2, -h/2, d/2);
glVertex3f(-w/2, h/2, d/2);
glVertex3f(-w/2, h/2, -d/2);
glVertex3f(-w/2, -h/2, -d/2);
glEnd();

// Right wall
glBegin(GL_QUADS);
glVertex3f(w/2, -h/2, d/2);
glVertex3f(w/2, -h/2, -d/2);
```

```
    glVertex3f(w/2, h/2, -d/2);
    glVertex3f(w/2, h/2, d/2);
    glEnd();

    // Simple airflow visualization (curved lines)
    if(climateMode == 1) {
        glColor3f(1.0f, 0.3f, 0.2f); // Hot mode - red airflow
    } else if(climateMode == 2) {
        glColor3f(0.2f, 0.4f, 0.8f); // Cold mode - blue
    } else {
        glColor3f(0.3f, 0.7f, 0.5f);
    }

    glLineWidth(2.0f);
    for(int i = 0; i < 5; i++) {
        float offset = (i - 2) * 10.0f;
        glBegin(GL_LINE_STRIP);
        for(float t = -h/2; t <= h/2; t += 2.0f) {
            float x = offset + sin(t * 0.1f) * 3.0f;
            glVertex3f(x, t, 0);
        }
        glEnd();
    }

    if(lightEnabled) glEnable(GL_LIGHTING);
    glPopMatrix();
}

// Draw Micro Vents
void drawMicroVents() {
    glPushMatrix();
    if(explodedView) {
        glTranslatef(0, explosionDistance * 0.0f, explosionDistance * 1.0f);
    } else {
```

```
    glTranslatef(0, 0, -5);
}

glColor3f(0.4f, 0.4f, 0.45f);

// Top vents
glPushMatrix();
glTranslatef(0, 39, 0);
for(int i = 0; i < 8; i++) {
    glPushMatrix();
    glTranslatef((i - 3.5f) * 7, 0, 0);
    glRotatef(ventAngle, 1, 0, 0);
    glBegin(GL_QUADS);
    glVertex3f(-3, 0, -1);
    glVertex3f(3, 0, -1);
    glVertex3f(3, 0, 1);
    glVertex3f(-3, 0, 1);
    glEnd();
    glPopMatrix();
}
glPopMatrix();

// Bottom vents
glPushMatrix();
glTranslatef(0, -39, 0);
for(int i = 0; i < 8; i++) {
    glPushMatrix();
    glTranslatef((i - 3.5f) * 7, 0, 0);
    glRotatef(ventAngle, 1, 0, 0);
    glBegin(GL_QUADS);
    glVertex3f(-3, 0, -1);
    glVertex3f(3, 0, -1);
    glVertex3f(3, 0, 1);
    glVertex3f(-3, 0, 1);
```

```
        glEnd();
        glPopMatrix();
    }
    glPopMatrix();

    glPopMatrix();
}

// Draw Mini Cooling Fan with metal texture
void drawMiniCoolingFan(bool isTop, float angle) {
    glPushMatrix();

    float yPos = isTop ? 38 : -38;
    float explodeOffset = isTop ? explosionDistance * 1.0f : -explosionDistance * 1.0f;

    if(explodedView) {
        glTranslatef(0, explodeOffset, explosionDistance * 0.5f);
    } else {
        glTranslatef(0, yPos, -5);
    }

    // Fan housing
    glPushMatrix();
    glEnable(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, metalTexture);
    glColor3f(0.8f, 0.8f, 0.85f);

    // Front face
    glBegin(GL_QUADS);
    glNormal3f(0, 0, 1);
    glTexCoord2f(0, 0); glVertex3f(-8, -4, 4);
    glTexCoord2f(1, 0); glVertex3f(8, -4, 4);
    glTexCoord2f(1, 1); glVertex3f(8, 4, 4);
    glTexCoord2f(0, 1); glVertex3f(-8, 4, 4);
```

```
glEnd();

// Back face
glBegin(GL_QUADS);
glNormal3f(0, 0, -1);
glTexCoord2f(0, 0); glVertex3f(-8, -4, -4);
glTexCoord2f(1, 0); glVertex3f(8, -4, -4);
glTexCoord2f(1, 1); glVertex3f(8, 4, -4);
glTexCoord2f(0, 1); glVertex3f(-8, 4, -4);
glEnd();

// Side faces
glBegin(GL_QUADS);
glNormal3f(0, 1, 0);
glTexCoord2f(0, 0); glVertex3f(-8, 4, -4);
glTexCoord2f(1, 0); glVertex3f(8, 4, -4);
glTexCoord2f(1, 1); glVertex3f(8, 4, 4);
glTexCoord2f(0, 1); glVertex3f(-8, 4, 4);
glEnd();

glBegin(GL_QUADS);
glNormal3f(0, -1, 0);
glTexCoord2f(0, 0); glVertex3f(-8, -4, -4);
glTexCoord2f(0, 1); glVertex3f(-8, -4, 4);
glTexCoord2f(1, 1); glVertex3f(8, -4, 4);
glTexCoord2f(1, 0); glVertex3f(8, -4, -4);
glEnd();

glBegin(GL_QUADS);
glNormal3f(-1, 0, 0);
glTexCoord2f(0, 0); glVertex3f(-8, -4, -4);
glTexCoord2f(1, 0); glVertex3f(-8, 4, -4);
glTexCoord2f(1, 1); glVertex3f(-8, 4, 4);
glTexCoord2f(0, 1); glVertex3f(-8, -4, 4);
```

```
glEnd();

glBegin(GL_QUADS);
glNormal3f(1, 0, 0);
glTexCoord2f(0, 0); glVertex3f(8, -4, -4);
glTexCoord2f(0, 1); glVertex3f(8, -4, 4);
glTexCoord2f(1, 1); glVertex3f(8, 4, 4);
glTexCoord2f(1, 0); glVertex3f(8, 4, -4);
glEnd();

glDisable(GL_TEXTURE_2D);

// Add ventilation grill pattern on front
glDisable(GL_LIGHTING);
glColor3f(0.15f, 0.15f, 0.18f);
glLineWidth(2.0f);
for(int i = 1; i < 7; i++) {
   float yOffset = -4.0f + i * (8.0f / 7.0f);
   glBegin(GL_LINES);
   glVertex3f(-7.5f, yOffset, 4.01f);
   glVertex3f(7.5f, yOffset, 4.01f);
   glEnd();
}

for(int i = 1; i < 5; i++) {
   float xOffset = -8.0f + i * (16.0f / 5.0f);
   glBegin(GL_LINES);
   glVertex3f(xOffset, -3.5f, 4.01f);
   glVertex3f(xOffset, 3.5f, 4.01f);
   glEnd();
}

if(lightEnabled) glEnable(GL_LIGHTING);
glPopMatrix();
```

```
// Fan blades
glPushMatrix();
glTranslatef(0, 0, -1);
glRotatef(angle, 0, 0, 1);
glDisable(GL_LIGHTING);

for(int i = 0; i < 4; i++) {
   glPushMatrix();
   glRotatef(i * 90, 0, 0, 1);

   glColor3f(0.7f, 0.7f, 0.75f);
   glBegin(GL_QUADS);
   glVertex3f(0.5f, -1.5f, 0.3f);
   glVertex3f(5.5f, -1.2f, 0.3f);
   glVertex3f(6.0f, 1.2f, 0.3f);
   glVertex3f(1.0f, 1.5f, 0.3f);
   glEnd();

   glColor3f(0.5f, 0.5f, 0.55f);
   glBegin(GL_QUADS);
   glVertex3f(0.5f, -1.5f, 0.3f);
   glVertex3f(0.5f, -1.5f, -0.3f);
   glVertex3f(5.5f, -1.2f, -0.3f);
   glVertex3f(5.5f, -1.2f, 0.3f);
   glEnd();

   glBegin(GL_QUADS);
   glVertex3f(5.5f, -1.2f, 0.3f);
   glVertex3f(5.5f, -1.2f, -0.3f);
   glVertex3f(6.0f, 1.2f, -0.3f);
   glVertex3f(6.0f, 1.2f, 0.3f);
   glEnd();
```

```
        glColor3f(0.6f, 0.6f, 0.65f);
        glBegin(GL_QUADS);
        glVertex3f(0.5f, -1.5f, -0.3f);
        glVertex3f(1.0f, 1.5f, -0.3f);
        glVertex3f(6.0f, 1.2f, -0.3f);
        glVertex3f(5.5f, -1.2f, -0.3f);
        glEnd();

        glPopMatrix();
    }

    glColor3f(0.3f, 0.3f, 0.35f);
    GLUquadric* quad = gluNewQuadric();
    gluCylinder(quad, 1.5, 1.5, 1.2, 16, 1);
    glPushMatrix();
    glTranslatef(0, 0, 1.2);
    gluDisk(quad, 0, 1.5, 16, 1);
    glPopMatrix();
    gluDeleteQuadric(quad);

    if(lightEnabled) glEnable(GL_LIGHTING);
    glPopMatrix();

    glPopMatrix();
}

// Draw Inner Insulated Glass Layer
void drawInnerInsulatedGlass() {
    glPushMatrix();
    if(explodedView) {
        glTranslatef(0, 0, -explosionDistance * 0.8f);
    } else {
        glTranslatef(0, 0, -10);
    }
```

```
    drawGlassPanel(60, 80, 1.5f, 0.7f, 0.75f, 0.8f, 0.5f, true);
    glPopMatrix();
}

// Draw sensor module
void drawSensor(float x, float y, float z, float r, float g, float b) {
    glPushMatrix();
    glTranslatef(x, y, z);
    glColor3f(r, g, b);
    glutSolidSphere(1.5, 16, 16);
    glPopMatrix();
}

// Draw Intelligent Frame System with metal texture
void drawIntelligentFrame() {
    glPushMatrix();
    if(explodedView) {
        glScalef(1.0f + explosionDistance * 0.01f, 1.0f + explosionDistance * 0.01f, 1.0f);
    }

    glEnable(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, metalTexture);
    glColor3f(0.8f, 0.8f, 0.85f);

    float h = 84, d = 15, thick = 4;

    // Top frame
    glPushMatrix();
    glTranslatef(0, h/2, -d/2);
    glScalef(68, thick, d);
    glutSolidCube(1);
    glPopMatrix();

    // Bottom frame
```

```
glPushMatrix();
glTranslatef(0, -h/2, -d/2);
glScalef(68, thick, d);
glutSolidCube(1);
glPopMatrix();

// Left frame
glPushMatrix();
glTranslatef(-64/2, 0, -d/2);
glScalef(thick, h, d);
glutSolidCube(1);
glPopMatrix();

// Right frame
glPushMatrix();
glTranslatef(64/2, 0, -d/2);
glScalef(thick, h, d);
glutSolidCube(1);
glPopMatrix();

glDisable(GL_TEXTURE_2D);

// Sensors
if(climateMode == 1) {
   drawSensor(32.5, 0, 1, 1.0f, 0.2f, 0.2f); // Hot - red
} else if(climateMode == 2) {
   drawSensor(32.5, 0, 1, 0.2f, 0.4f, 1.0f); // Cold - blue
} else {
   drawSensor(32.5, 0, 1, 0.3f, 1.0f, 0.3f); // Normal - green
}

// Light sensor
drawSensor(32.5, 40, 1, 0.9f, 0.9f, 0.3f);
```

```
// Microcontroller box
glPushMatrix();
glTranslatef(28, -35, 0);
glColor3f(0.15f, 0.15f, 0.2f);
glScalef(6, 8, 4);
glutSolidCube(1);
glPopMatrix();

// Display panel with texture
glPushMatrix();
glTranslatef(-28, -38, 2);
glEnable(GL_TEXTURE_2D);
glBindTexture(GL_TEXTURE_2D, displayTexture);
glColor3f(1.0f, 1.0f, 1.0f);
glBegin(GL_QUADS);
glNormal3f(0, 0, 1);
glTexCoord2f(0, 0); glVertex3f(-3, -2, 0);
glTexCoord2f(1, 0); glVertex3f(3, -2, 0);
glTexCoord2f(1, 1); glVertex3f(3, 2, 0);
glTexCoord2f(0, 1); glVertex3f(-3, 2, 0);
glEnd();
glDisable(GL_TEXTURE_2D);

// LED indicators with glow effect
glDisable(GL_LIGHTING);
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE);

// Pulsating glow effect
float glowIntensity = 0.5f + 0.5f * sin(animationTime * 3.0f);

glColor4f(0.0f, 1.0f, 0.0f, glowIntensity * 0.8f);
glPointSize(8.0f);
glBegin(GL_POINTS);
```

```
    glVertex3f(-1, 0, 0.1f);
    glVertex3f(1, 0, 0.1f);
    glEnd();

    glColor4f(0.0f, 1.0f, 0.0f, glowIntensity * 0.3f);
    glPointSize(12.0f);
    glBegin(GL_POINTS);
    glVertex3f(-1, 0, 0.1f);
    glVertex3f(1, 0, 0.1f);
    glEnd();

    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    if(lightEnabled) glEnable(GL_LIGHTING);
    glPopMatrix();

    glPopMatrix();
}

void drawCompleteIWS() {
    drawIntelligentFrame();
    drawOuterSmartGlass();
    drawSolarCoating();
    drawAirflowChamber();
    drawMicroVents();
    drawMiniCoolingFan(true, topFanAngle);
    drawMiniCoolingFan(false, bottomFanAngle);
    drawInnerInsulatedGlass();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    gluLookAt(0, 0, cameraDistance, 0, 0, 0, 0, 1, 0);
```

```
glRotatef(xRotation, 1, 0, 0);
glRotatef(yRotation, 0, 1, 0);
glRotatef(zRotation, 0, 0, 1);

drawCompleteIWS();

// Draw text info
glDisable(GL_LIGHTING);
glMatrixMode(GL_PROJECTION);
glPushMatrix();
glLoadIdentity();
gluOrtho2D(0, 800, 0, 600);
glMatrixMode(GL_MODELVIEW);
glPushMatrix();
glLoadIdentity();

glColor3f(1.0f, 1.0f, 1.0f);
glRasterPos2f(10, 580);
char info[100];
sprintf(info, "Mode: %s | Exploded: %s | Press 'H' for help",
      climateMode == 1 ? "HOT" : (climateMode == 2 ? "COLD" : "NORMAL"),
      explodedView ? "YES" : "NO");
for(char* c = info; *c != '\0'; c++) {
   glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, *c);
}

glPopMatrix();
glMatrixMode(GL_PROJECTION);
glPopMatrix();
glMatrixMode(GL_MODELVIEW);

if(lightEnabled) glEnable(GL_LIGHTING);
```

```
    glutSwapBuffers();
}

// Timer callback for animations
void timer(int value) {
    animationTime += 0.016f; // ~16ms per frame

    // Fan animations
    if(topFanRotate) {
        topFanAngle += 12.0f * fanSpeedMultiplier;
        if(topFanAngle > 360.0f) topFanAngle -= 360.0f;
    }

    if(bottomFanRotate) {
        bottomFanAngle += 12.0f * fanSpeedMultiplier;
        if(bottomFanAngle > 360.0f) bottomFanAngle -= 360.0f;
    }

    // Vent animation
    if(ventAnimate) {
        if(ventOpening) {
            ventAngle += 2.0f;
            if(ventAngle >= 90.0f) {
                ventAngle = 90.0f;
                ventOpening = false;
            }
        } else {
            ventAngle -= 2.0f;
            if(ventAngle <= 0.0f) {
                ventAngle = 0.0f;
                ventOpening = true;
            }
        }
    }
```

```
    glutPostRedisplay();
    glutTimerFunc(16, timer, 0);
}

// Apply climate mode settings
void applyClimateMode(int mode) {
    climateMode = mode;

    if(mode == 1) { // HOT
        glassTintLevel = 0.8f;
        ventAngle = 90.0f;
        topFanRotate = true;
        bottomFanRotate = true;
        fanSpeedMultiplier = 2.0f;
    } else if(mode == 2) { // COLD
        glassTintLevel = 0.1f;
        ventAngle = 0.0f;
        topFanRotate = false;
        bottomFanRotate = false;
        fanSpeedMultiplier = 0.0f;
    } else { // NORMAL
        glassTintLevel = 0.4f;
        ventAngle = 45.0f;
        topFanRotate = true;
        bottomFanRotate = true;
        fanSpeedMultiplier = 1.0f;
    }
}

// Keyboard callback
void keyboard(unsigned char key, int x, int y) {
    switch(key) {
        case 27: exit(0); break;
        case 'z': case 'Z':
```

```
    cameraDistance -= 5.0f;
    if(cameraDistance < 50.0f) cameraDistance = 50.0f;
    break;
case 'x': case 'X':
    cameraDistance += 5.0f;
    if(cameraDistance > 300.0f) cameraDistance = 300.0f;
    break;
case 'a': case 'A': zRotation += 5.0f; break;
case 'd': case 'D': zRotation -= 5.0f; break;
case 'l': case 'L':
    lightEnabled = true;
    glEnable(GL_LIGHTING);
    printf("Light enabled\n");
    break;
case 'k': case 'K':
    lightEnabled = false;
    glDisable(GL_LIGHTING);
    printf("Light disabled\n");
    break;
case 'f': case 'F': topFanRotate = !topFanRotate; break;
case 'g': case 'G': bottomFanRotate = !bottomFanRotate; break;
case 'v': case 'V': ventAnimate = !ventAnimate; break;
case 't': case 'T':
    glassTintLevel += 0.1f;
    if(glassTintLevel > 1.0f) glassTintLevel = 1.0f;
    break;
case 'y': case 'Y':
    glassTintLevel -= 0.1f;
    if(glassTintLevel < 0.0f) glassTintLevel = 0.0f;
    break;
case '1':
    applyClimateMode(1); // HOT
    printf("Mode 1 (HOT): Dark glass, open vents, fast fans\n");
    break;
```

```
case '2':
    applyClimateMode(2); // COLD
    printf("Mode 2 (COLD): Clear glass, closed vents, fans off\n");
    break;
case '3':
    applyClimateMode(3); // NORMAL
    printf("Mode 3 (NORMAL): Balanced settings\n");
    break;
case 'e': case 'E':
    explodedView = !explodedView;
    explosionDistance = explodedView ? maxExplosion : 0.0f;
    printf("Exploded view: %s\n", explodedView ? "ON" : "OFF");
    break;
case '+': case '=':
    explosionDistance += 5.0f;
    if(explosionDistance > 150.0f) explosionDistance = 150.0f;
    if(explosionDistance > 0) explodedView = true;
    break;
case '-': case '_':
    explosionDistance -= 5.0f;
    if(explosionDistance < 0.0f) explosionDistance = 0.0f;
    if(explosionDistance == 0) explodedView = false;
    break;
case 'h': case 'H':
    printf("\n=== ENHANCED IWS CONTROLS ===\n");
    printf("Arrow Keys: Rotate view\n");
    printf("Z/X: Camera zoom\n");
    printf("A/D: Rotate around Z-axis\n");
    printf("L/K: Light on/off\n");
    printf("F: Toggle top fan\n");
    printf("G: Toggle bottom fan\n");
    printf("V: Animate vents\n");
    printf("T/Y: Adjust glass tint\n");
    printf("1: HOT mode\n");
```

```
            printf("2: COLD mode\n");
            printf("3: NORMAL mode\n");
            printf("E: Toggle exploded view\n");
            printf("+/=: Increase explosion distance\n");
            printf("-/_: Decrease explosion distance\n");
            printf("ESC: Exit\n");
            break;
    }
    glutPostRedisplay();
}

// Special keys callback
void specialKeys(int key, int x, int y) {
    switch(key) {
        case GLUT_KEY_UP: xRotation += 5.0f; break;
        case GLUT_KEY_DOWN: xRotation -= 5.0f; break;
        case GLUT_KEY_LEFT: yRotation -= 5.0f; break;
        case GLUT_KEY_RIGHT: yRotation += 5.0f; break;
    }
    glutPostRedisplay();
}

// Mouse callback
void mouse(int button, int state, int x, int y) {
    if(button == GLUT_LEFT_BUTTON) {
        if(state == GLUT_DOWN) {
            isMouseDragging = true;
            lastMouseX = x;
            lastMouseY = y;
        } else {
            isMouseDragging = false;
        }
    }
}
```

```
// Mouse motion callback
void mouseMotion(int x, int y) {
    if(isMouseDragging) {
        int dx = x - lastMouseX;
        int dy = y - lastMouseY;
        yRotation += dx * 0.5f;
        xRotation += dy * 0.5f;
        lastMouseX = x;
        lastMouseY = y;
        glutPostRedisplay();
    }
}

void reshape(int w, int h) {
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double)w / (double)h, 1.0, 500.0);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Intelligent Window System (IWS) – Adaptive Climate & Light Control");

    init();

    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
```

```
    glutKeyboardFunc(keyboard);
    glutSpecialFunc(specialKeys);
    glutMouseFunc(mouse);
    glutMotionFunc(mouseMotion);
    glutTimerFunc(16, timer, 0);

    printf("=== Intelligent Window System (IWS) ===\n");
    printf("Press 'H' for help\n");
    printf("\nStarting in NORMAL mode...\n\n");

    applyClimateMode(3); // Start in normal mode

    glutMainLoop();
    return 0;
}
```